Testes Unitários com JUnit

Prof. Msc Marcos de Souza

Mas antes ...

FACULDADE IBRATEC

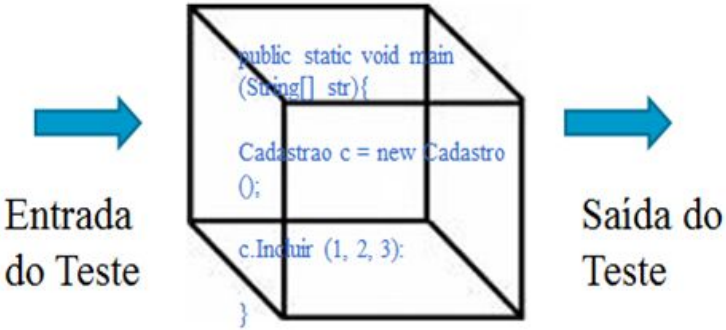
Mas antes ...

- Como podemos garantir que uma determinada funcionalidade esteja com o comportamento esperado ?
- Como realizar essa garantia de forma automática ?
- Como mensurar a qualidade do código (cobertura) ?

Mas antes ...

Para resolver tais questionamentos é preciso a realização de testes de software:

- Mas o que são testes?



Tipos de Testes

Existem dois tipos de testes:

- Testes caixa preta
- Testes caixa branca

Testes Caixa Branca

- São testes realizados, geralmente, pela equipe de desenvolvimento do software;
- É possível ter acesso ao código do software;

```
@Test
public void multiplicarNegativos() {
    System.out.println("multiplicar negativos");
    int n1 = -2;
    int n2 = -3;
    Calculadora instance = new Calculadora();
    double expResult = 6;
    double result = instance.multiplicar(n1, n2);
    assertEquals(expResult, result, 0);
}
```

Testes Caixa Preta

- São testes realizados sobre o sistema em funcionamento;
- Geralmente é realizado em um esquema de passo-a-passo;
- Não é possível ter acesso ao código apenas à UI.

Testes Funcionais

- Os testes geralmente são norteados pelos casos de testes:

Ação	Resultado Esperado
Preencher o campo “CPF” com um valor inválido.	
Pressionar a tecla Tab.	Exibida mensagem informando que o CPF é inválido.

Ação	Resultado Esperado
Preencher o campo “CPF” com um valor inválido e pressionar a tecla Tab.	Exibida mensagem informando que o CPF é inválido.

Casos de Testes

- Também é possível encontrar o seguinte template para os casos de testes:

Test Case Tradicional	
ID	TCS-123
Nome	Consultar um Triângulo Escaleno
Ambiente	RVM-1.8.7
Ator	Aluno de Matemática
Pré-Condições	Não existir nenhum triângulo
Procedimentos (Entradas e Saídas)	1 – Abra a aplicação 2 – Solicite consultar o tipo de um triângulo 3 – O sistema solicita os tamanhos dos lados do triângulo 4 – Informe o lado A como “3“ 5 – Informe o lado B como “4“ 6 – Informe o lado C como “5“ 7 – O sistema informa que o triângulo é “Escaleno”
Pós-Condições	Existe um triângulo do tipo escaleno

JUnit

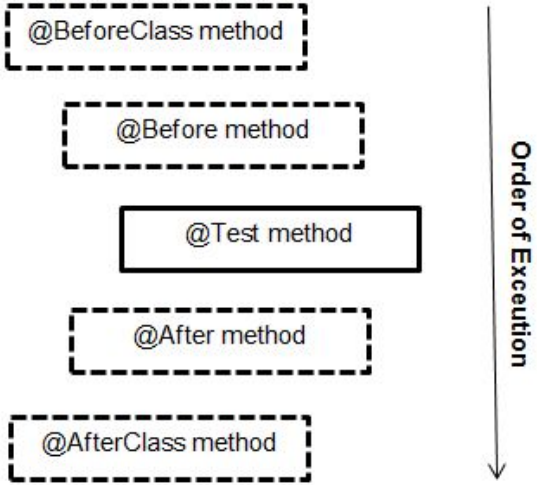
FACULDADE IBRATEC

Testes Unitários

- Testes unitários são testes do tipo caixa branca que visam validar uma ‘unidade’;
- A unidade pode ser: método; classe; funcionalidade; etc.
- Em Java a biblioteca amplamente utilizada é a JUnit;
- Porém existem diversos frameworks para a automação de testes, por exemplo: Selenium para web; UIAutomator para Android, etc.

Testes Unitários

- Uma vez o caso de teste estando definido é possível automatizar com JUnit através das seguintes annotations:



Passo a passo

FACULDADE IBRATEC

Dependência

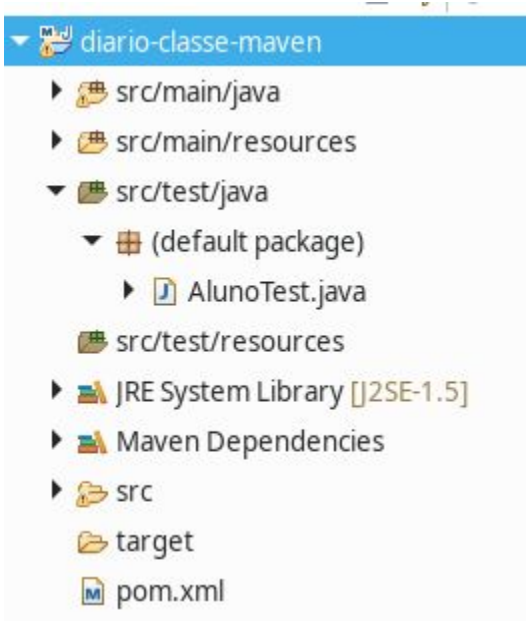
- Primeiro passo, em um projeto MAVEN, é adicionar a dependência do JUnit:

Maven | Gradle | SBT | Ivy | Grape | Leiningen | Buildr

```
<!-- https://mvnrepository.com/artifact/junit/junit -->
<dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.12</version>
    <scope>test</scope>
</dependency>
```

Classe de teste

- Em seguida criar a classe de teste em src/test/java:



Métodos de Testes

- Por fim devemos criar nossos métodos de teste;
- Quando terminar a escrita a execução se dará, ao clicar com o **botão direito > Run As > JUnit Test**

```java
1  import static org.junit.Assert.assertEquals;
2
3  import org.junit.Test;
4
5  public class AlunoTest {
6
7      @Test
8      public void testFailExample() {
9          int a = 1;
10         int b = 2;
11
12         assertEquals(a, b);
13     }
14
15     @Test
16     public void testSuccessExample() {
17         int a = 1;
18         int b = 1;
19
20         assertEquals(a, b);
21     }
22
23 }
24
```

Problems | Javadoc | Declaration | Progress | Console | JUnit

Finished after 0,031 seconds

Runs: 2/2 | Errors: 0 | Failures: 1

- AlunoTest [Runner: JUnit 4] (0,005 s)
  - testFailExample (0,003 s)
  - testSuccessExample (0,001 s)

Failure Trace

```
java.lang.AssertionError: expected:<1> but was:<2>
at AlunoTest.testFailExample(AlunoTest.java:12)
```

Métodos de Testes

- Caso se deseje ignorar a execução de um determinado teste deve-se utilizar a annotation @ignore;

Exemplo prático

- Considere o seguinte método:

```
public boolean inserir(Aluno aluno) {
    boolean result = false;

    try {

        if (aluno != null) {
            dao.inserir(aluno);
            result = true;
        }
    } catch (Exception e) {
        System.out.println("Houve um erro");
    }

    return result;
}
```

Exemplo prático

- Observando o método um exemplo bem claro é passar um objeto nulo (não precisaria depender do acesso ao DB), onde o retorno deverá ser falso;

```
@Test
public void testInserirAlunoInvalido() {
    AlunoController controller = new AlunoController();

    boolean resultadoEsperado = false;
    boolean resultado = controller.inserir(null);

    assertEquals(resultadoEsperado, resultado);
}
```

Algumas observações

- Os testes precisam ser autocontidos, ou seja não podem ter dependências daquilo que não se está validando. No exemplo prático, não é preciso que o banco esteja no ar para validar o controller de adicionar um aluno;
- Os métodos de teste, por convenção, iniciam o nome com a palavra ‘test’ seguido de um resumo a que se trata o teste;
- Os métodos de teste são públicos e não possuem retorno (void);
- Uma boa prática é iniciar o desenvolvimento a partir da criação dos testes (TDD).